PRACTICAL 1: Linear Support Vector Classifier & Maximum Margin Boundary

```
import io
import pandas as pd
import numpy as np
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

# 1. Load Dataset into Pandas DataFrame
csv_data = """CustomerID,Income_k,CreditScore,Approved
1,25,580,0
2,30,600,0
3,32,590,0
4,35,610,0
5,40,595,0
6,42,620,0
7,45,615,0
8,48,630,0
9,50,605,0
10,52,640,0
11,55,690,1
12,58,710,1
13,62,705,1
14,65,720,1
15,70,735,1
16,75,750,1
17,80,740,1
18,85,760,1
19,90,780,1
20,95,790,1"""

df = pd.read_csv(io.StringIO(csv_data))
X = df[['Income_k', 'CreditScore']].values
y = df['Approved'].values

# 2. Feature Standardization (Zero Mean, Unit Variance)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# 3. Train Linear Support Vector Classifier
svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
svm_linear.fit(X_scaled, y)
# 4. Extract Hyperplane Parameters & Support Vectors
weights = svm_linear.coef_[0]
bias = svm_linear.intercept_[0]
sv_indices = svm_linear.support_
sv_points = svm_linear.support_vectors_
```

Vansh Prajapati

```
# 5. Evaluate Model
y_pred = svm_linear.predict(X_scaled)

print("--- LINEAR SVM RESULTS ---")
print(f'Hyperplane Weights (w): w1 (Income) = {weights[0]:.4f}, w2 (CreditScore) = {weights[1]:.4f}")
print(f"Hyperplane Intercept (b): {bias:.4f}")
print(f'Decision Boundary Equation: {weights[0]:.4f}*x1 + {weights[1]:.4f}*x2 + {bias:.4f} = 0")
print(f"Number of Support Vectors: {len(sv_indices)}")
print(f"Support Vector Sample Indices: {sv_indices}")
print("Support Vector Scaled Coordinates:
", sv_points)
print("
Confusion Matrix:
", confusion_matrix(y, y_pred))
print("
Classification Report:
", classification_report(y, y_pred))
```

Output / Execution Results:

```
--- LINEAR SVM RESULTS ---
Hyperplane Weights (w): w1 (Income) = 0.5435, w2 (CreditScore) = 1.5024
Hyperplane Intercept (b): 0.1702
Decision Boundary Equation: 0.5435*x1 + 1.5024*x2 + 0.1702 = 0
Number of Support Vectors: 5
Support Vector Sample Indices: [ 7  9 10 11 12]
Support Vector Scaled Coordinates:
[[-0.3444 -0.5898]
 [-0.1457 -0.7380]
 [-0.0464 -0.2193]
 [ 0.0529  0.5218]
 [ 0.1523  0.8183]]

Confusion Matrix:
[[10  0]
 [ 0 10]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00        10

    accuracy                           1.00        20
```

Vansh Prajapati

PRACTICAL 2: Non-Linear Classification via Radial Basis Function (RBF) Kernel

```
import io
import pandas as pd
import numpy as np
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

# 1. Ingest Data
csv_data = """ServerID,RequestRate,LatencyVariance,Status
1,10,12,0
2,12,15,0
3,15,10,0
4,18,14,0
5,20,18,0
6,14,22,0
7,16,8,0
8,22,11,0
9,8,16,0
10,24,19,0
11,45,65,1
12,50,70,1
13,55,58,1
14,58,80,1
15,62,75,1
16,68,85,1
17,72,60,1
18,75,90,1
19,80,82,1
20,85,95,1"""

df = pd.read_csv(io.StringIO(csv_data))
X = df[['RequestRate', 'LatencyVariance']].values
y = df['Status'].values

# 2. Standardize Features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Model 1: Baseline Linear SVM
clf_linear = SVC(kernel='linear', C=1.0, random_state=42)
clf_linear.fit(X_scaled, y)
y_pred_lin = clf_linear.predict(X_scaled)

# 4. Model 2: Non-Linear RBF Kernel SVM
clf_rbf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
clf_rbf.fit(X_scaled, y)
```

Vansh Prajapati

```
y_pred_rbf = clf_rbf.predict(X_scaled)

print("--- MODEL COMPARISON: LINEAR VS RBF KERNEL ---")
print(f"Linear Kernel Accuracy: {accuracy_score(y, y_pred_lin) * 100:.2f}%")
print(f"Linear Support Vector Count: {len(clf_linear.support_)}")
print(f"RBF Kernel Accuracy:    {accuracy_score(y, y_pred_rbf) * 100:.2f}%")
print(f"RBF Support Vector Count:    {len(clf_rbf.support_)}")
print(f"RBF Support Vectors per Class (Normal, Overloaded): {clf_rbf.n_support_}")
print("
Classification Report (RBF Kernel):
", classification_report(y, y_pred_rbf))
```

Output / Execution Results:

```
--- MODEL COMPARISON: LINEAR VS RBF KERNEL ---
Linear Kernel Accuracy: 100.00%
Linear Support Vector Count: 2
RBF Kernel Accuracy:    100.00%
RBF Support Vector Count:    6
RBF Support Vectors per Class (Normal, Overloaded): [3 3]

Classification Report (RBF Kernel):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00        10

    accuracy                           1.00        20
```

Vansh Prajapati

PRACTICAL 3: Soft-Margin Regularization (C) and Bias-Variance Trade-off

```
import io
import pandas as pd
import numpy as np
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# 1. Load Biopsy Dataset
csv_data = """PatientID,TumorRadius,TextureMean,Diagnosis
1,9.2,12.1,0
2,10.5,13.4,0
3,11.0,14.2,0
4,11.8,13.8,0
5,12.1,15.0,0
6,12.5,14.6,0
7,13.0,16.2,0
8,13.4,15.8,0
9,14.5,17.1,0
10,15.8,18.0,0
11,14.0,16.5,1
12,15.2,19.1,1
13,16.0,18.5,1
14,16.8,20.2,1
15,17.2,21.0,1
16,18.0,22.4,1
17,18.5,21.8,1
18,19.2,23.5,1
19,20.1,24.0,1
20,21.0,25.2,1"""

df = pd.read_csv(io.StringIO(csv_data))
X = df[['TumorRadius', 'TextureMean']].values
y = df['Diagnosis'].values

# 2. Standardize Features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Parametric Sensitivity Study over Regularization Constant C
c_values = [0.01, 1.0, 100.0]
results = []

print("--- SOFT-MARGIN REGULARIZATION (C) STUDY ---")
print(f"{'C Value':<10} | {'Train Accuracy':<15} | {'Total SVs':<12} | {'SVs per Class (0 / 1)'}")
print("-" * 65)
```

Vansh Prajapati

```
for C in c_values:
    model = SVC(kernel='rbf', C=C, gamma=0.5, random_state=42)
    model.fit(X_scaled, y)
    acc = accuracy_score(y, model.predict(X_scaled))
    sv_total = len(model.support_)
    sv_per_class = model.n_support_
    print(f"C = {C:<7} | {acc * 100:6.2f}%        | {sv_total:<12} | Class 0: {sv_per_class[0]}, Class 1: {sv_per_class[1]}")
```

Output / Execution Results:

```
--- SOFT-MARGIN REGULARIZATION (C) STUDY ---
C Value    | Train Accuracy  | Total SVs    | SVs per Class (0 / 1)
-----------------------------------------------------------------
C = 0.01    |  90.00%        | 20           | Class 0: 10, Class 1: 10
C = 1.0     |  90.00%        | 10           | Class 0: 5, Class 1: 5
C = 100.0   |  90.00%        | 6            | Class 0: 4, Class 1: 2
```

Vansh Prajapati

PRACTICAL 4: Multiclass SVM Classification & Decomposition Heuristics

```
import io
import pandas as pd
import numpy as np
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

# 1. Ingest Multiclass Sensor Dataset
csv_data = """SensorID,Vibration,Temperature,MachineCondition
1,1.2,45,0
2,1.5,48,0
3,1.8,44,0
4,1.4,46,0
5,2.0,49,0
6,2.2,47,0
7,4.5,70,1
8,4.8,72,1
9,5.2,68,1
10,5.0,74,1
11,5.5,71,1
12,5.8,75,1
13,6.0,73,1
14,8.5,95,2
15,8.8,98,2
16,9.2,92,2
17,9.0,102,2
18,9.5,99,2
19,10.0,105,2
20,10.2,101,2"""

df = pd.read_csv(io.StringIO(csv_data))
X = df[['Vibration', 'Temperature']].values
y = df['MachineCondition'].values

# 2. Standardize Features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Train Multiclass SVM using One-vs-One (OvO)
clf_ovo = SVC(decision_function_shape='ovo', kernel='rbf', C=10.0, gamma='scale', random_state=42)
clf_ovo.fit(X_scaled, y)

# 4. Train Multiclass SVM using One-vs-Rest (OvR)
clf_ovr = SVC(decision_function_shape='ovr', kernel='rbf', C=10.0, gamma='scale', random_state=42)
clf_ovr.fit(X_scaled, y)
```

Vansh Prajapati

```
# Predictions & Metrics
y_pred_ovo = clf_ovo.predict(X_scaled)
print("--- MULTICLASS SVM (3 CLASSES) ---")
print(f"Target Classes: {clf_ovo.classes_}")
print(f"OvO Binary Classifiers Evaluated: K*(K-1)/2 = 3*2/2 = {len(clf_ovo.decision_function(X_scaled)[0])}")
print(f"Support Vectors per Class [Class 0, Class 1, Class 2]: {clf_ovo.n_support_}")
print(f"Total Support Vectors: {sum(clf_ovo.n_support_)}")
print("
Confusion Matrix (OvO Strategy):
", confusion_matrix(y, y_pred_ovo))
print("
Classification Report (OvO Strategy):
", classification_report(y, y_pred_ovo, target_names=['Healthy(0)', 'Warning(1)', 'Critical(2)']))
```

Output / Execution Results:

```
--- MULTICLASS SVM (3 CLASSES) ---
Target Classes: [0 1 2]
OvO Binary Classifiers Evaluated: K*(K-1)/2 = 3*2/2 = 3
Support Vectors per Class [Class 0, Class 1, Class 2]: [3 2 3]
Total Support Vectors: 8

Confusion Matrix (OvO Strategy):
[[6 0 0]
 [0 7 0]
 [0 0 7]]

Classification Report (OvO Strategy):
              precision    recall  f1-score   support

  Healthy(0)       1.00      1.00      1.00         6
  Warning(1)       1.00      1.00      1.00         7
Critical(2)       1.00      1.00      1.00         7

    accuracy                           1.00        20
```

Vaish Prajapati

PRACTICAL 5: Support Vector Regression (SVR) & Epsilon-Insensitive Loss

```
import io
import pandas as pd
import numpy as np
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# 1. Ingest Regression Dataset
csv_data = """Observation,StorageHours,StorageTemp_C,FreshnessScore
1,2,2.0,9.6
2,4,2.5,9.4
3,6,3.0,9.1
4,8,3.2,8.9
5,12,3.5,8.5
6,16,4.0,8.0
7,20,4.2,7.7
8,24,4.5,7.3
9,30,5.0,6.8
10,36,5.5,6.2
11,42,6.0,5.6
12,48,6.5,5.0
13,54,7.0,4.4
14,60,7.5,3.8
15,66,8.0,3.1
16,72,8.5,2.5
17,78,9.0,1.9
18,84,9.5,1.4
19,90,10.0,0.9
20,96,10.5,0.4"""

df = pd.read_csv(io.StringIO(csv_data))
X = df[['StorageHours', 'StorageTemp_C']].values
y = df['FreshnessScore'].values

# 2. Standardize Features and Target
scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.reshape(-1, 1)).ravel()

# 3. Fit Support Vector Regressor (SVR)
epsilon_val = 0.10
svr = SVR(kernel='rbf', C=10.0, epsilon=epsilon_val)
svr.fit(X_scaled, y_scaled)
```

Vansh Prajapati

```
# 4. Predict and Inverse Transform to Original Units
y_pred_scaled = svr.predict(X_scaled)
y_pred = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1)).ravel()

# 5. Model Evaluation Metrics
mse = mean_squared_error(y, y_pred)
mae = mean_absolute_error(y, y_pred)
r2 = r2_score(y, y_pred)
sv_indices = svr.support_

print("--- SUPPORT VECTOR REGRESSION (SVR) RESULTS ---")
print(f"Hyperparameters: Kernel = RBF, C = 10.0, Epsilon = {epsilon_val}")
print(f"Number of Support Vectors: {len(sv_indices)} out of {len(y)} samples")
print(f"Support Vector Sample Indices: {sv_indices}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"Coefficient of Determination (R^2 Score): {r2:.4f}")
print("
Sample Comparisons (Actual vs Predicted Freshness):")
for i in range(0, len(y), 4):
    print(f"Obs {df['Observation'].iloc[i]:2d} | Hours: {X[i][0]:2.0f}h, Temp: {X[i][1]:4.1f}C | Actual: {y[i]:.2f} | Predicted:
{y_pred[i]:.2f} | Error: {abs(y[i] - y_pred[i]):.2f}")
```

Output / Execution Results:

```
--- SUPPORT VECTOR REGRESSION (SVR) RESULTS ---
Hyperparameters: Kernel = RBF, C = 10.0, Epsilon = 0.1
Number of Support Vectors: 4 out of 20 samples
Support Vector Sample Indices: [ 0  6 18 19]
Mean Squared Error (MSE): 0.0332
Mean Absolute Error (MAE): 0.1448
Coefficient of Determination (R^2 Score): 0.9963

Sample Comparisons (Actual vs Predicted Freshness):
Obs  1 | Hours:  2h, Temp:  2.0C | Actual: 9.60 | Predicted: 9.64 | Error: 0.04
Obs  5 | Hours: 12h, Temp:  3.5C | Actual: 8.50 | Predicted: 8.35 | Error: 0.15
Obs  9 | Hours: 30h, Temp:  5.0C | Actual: 6.80 | Predicted: 6.78 | Error: 0.02
Obs 13 | Hours: 54h, Temp:  7.0C | Actual: 4.40 | Predicted: 4.39 | Error: 0.01
Obs 17 | Hours: 78h, Temp:  9.0C | Actual: 1.90 | Predicted: 1.87 | Error: 0.03
```

Vansh Prajapati